

OE-EC704A — Web Technology

Semester VII • 3L:0T:0P • 3 Credits

Module 3 Exam Notes: OOP Principles, Inheritance, Polymorphism, Packages & Interfaces

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	Core Object-Oriented Programming (OOP) Principles	2
2	Inheritance and Reuse Models	2
2.1	Composition vs. Inheritance	3
3	Polymorphism and Runtime Resolution	3
3.1	Method Overloading vs. Overriding	3
3.2	Dynamic Method Dispatch	4
4	Interfaces, Abstract Classes, and Access Modifiers	4
4.1	Abstract Class vs. Interface	4
4.2	Access Modifiers	4
5	Object Class and SOLID Design Principles	5
5.1	The Root Object Class	5
5.2	SOLID Design Principles	5

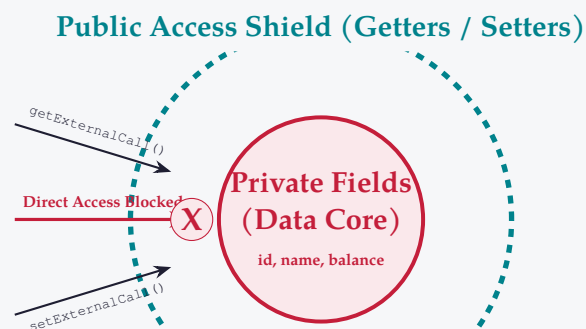
1. Core Object-Oriented Programming (OOP) Principles

Java is a pure object-oriented programming language designed around four fundamental concepts.

The Four Pillars of Object-Oriented Programming

1. **Encapsulation (Data Hiding):** Wrapping variables and methods together into a single unit (class) while restricting direct outside access using the `private` modifier and exposing access via `public` getters and setters.
2. **Abstraction:** Hiding internal execution complexity and showing only the essential feature interfaces to the outer user. Implemented using **Abstract Classes** and **Interfaces**.
3. **Inheritance:** The mechanism by which one class (subclass/child) inherits the state variables and functional behaviors of another class (superclass/parent) to facilitate code reusability.
4. **Polymorphism:** The ability of a reference to take on multiple forms. It is divided into:
 - **Compile-time (Static):** Resolved by compiler signatures via **Method Overloading**.
 - **Runtime (Dynamic):** Resolved at execution via **Method Overriding** and Dynamic Method Dispatch.

Encapsulation Boundary & Data Protection Shield



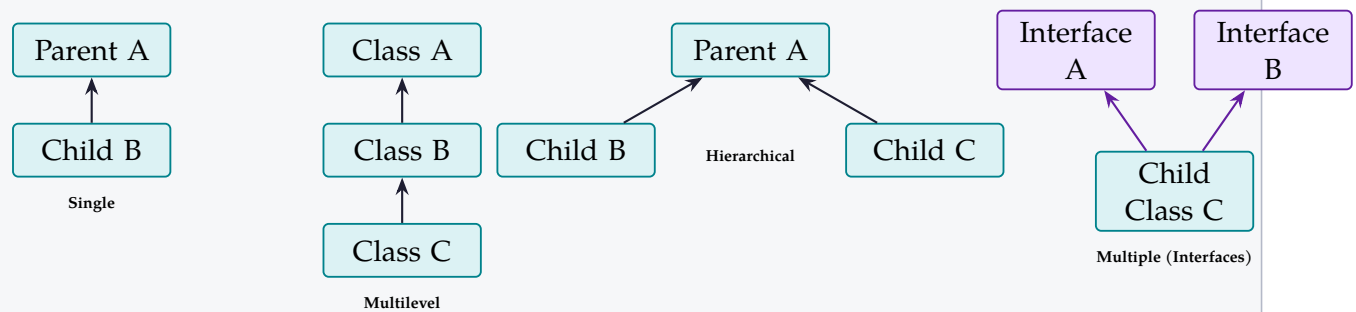
2. Inheritance and Reuse Models

Inheritance enables hierarchical classifications. In Java, classes are extended using the `extends` keyword.

Inheritance Characteristics

- **Single Inheritance Only:** Java classes can only extend **one** parent class to prevent the diamond problem (ambiguity under duplicate variable/method signatures).
- **The `super` Keyword:** Used to invoke the parent class constructor (must be the first statement in the subclass constructor), access overridden parent methods, or read hidden parent variables.

Inheritance Trees supported in Java



2.1 Composition vs. Inheritance

Design patterns advocate favoring object composition over class inheritance:

Metric	Composition (Has-A Relationship)	Inheritance (Is-A Relationship)
Coupling Level	Loose coupling; internal details remain private.	Tight coupling; breaks encapsulation boundary.
Flexibility	Behaviors can change dynamically at runtime.	Static; behavior fixed at compilation time.
Design Reuse	Reuses functional components as class members.	Reuses code by expanding a class hierarchy.
Example	Car has an Engine instance.	Car is a subclass of Vehicle.

3. Polymorphism and Runtime Resolution

3.1 Method Overloading vs. Overriding

These represent static and dynamic polymorphism:

Feature	Method Overloading (Static)	Method Overriding (Dynamic)
Class Context	Occurs within the same class definition.	Occurs across child-parent subclass hierarchy.
Method Signature	Name must be same; parameters must differ.	Signature (parameters, returns) must match.
Resolution Time	Compile-time (static binding).	Runtime (dynamic binding).
Keywords	None required.	Assisted by <code>@Override</code> annotation.

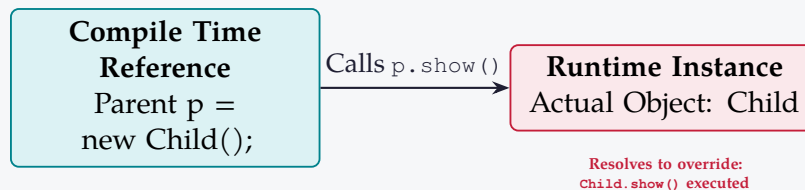
3.2 Dynamic Method Dispatch

Dynamic Method Dispatch is the mechanism by which a call to an overridden method is resolved at runtime rather than compile-time.

Dynamic Method Dispatch Rule

When an overridden method is called through a superclass reference, Java determines which version of the method to execute based on the **type of the actual object** being referred to at runtime, not the type of the reference variable.

Dynamic Method Dispatch Flow



4. Interfaces, Abstract Classes, and Access Modifiers

4.1 Abstract Class vs. Interface

These constructs provide different levels of abstraction:

Feature	Abstract Class	Interface
Multiple Inheritance	Allowed to inherit only one abstract parent.	A class can implement multiple interfaces.
State Variables	Can contain instance variables (stateful).	Variables must be <code>public static final</code> .
Methods	Concrete methods and abstract methods.	Abstract methods (pre-Java 8), defaults allowed.
Keywords	Declared with <code>abstract class</code> and <code>extends</code> .	Declared with <code>interface</code> and <code>implements</code> .

4.2 Access Modifiers

Java regulates visibility using four access levels:

- **private:** Visible only within the declaring class.
- **Default (no modifier):** Visible only to classes inside the same package.
- **protected:** Visible to the same package and subclass child files in external packages.
- **public:** Accessible from any class anywhere in the project.

5. Object Class and SOLID Design Principles

5.1 The Root Object Class

Every Java class implicitly extends `java.lang.Object`. It defines essential methods:

- **`toString()`**: Returns a string representation of the object (`ClassName@HexHashCode`).
- **`equals(Object obj)`**: Compares reference equality. Overridden to compare values.
- **`hashCode()`**: Returns an integer hash code mapped to the object's memory address.
- **`clone()`**: Creates a field-by-field copy of the current object.

5.2 SOLID Design Principles

These principles ensure Java classes are maintainable:

- **Single Responsibility Principle (SRP)**: A class must have exactly one reason to change.
- **Open/Closed Principle (OCP)**: Software entities should be open for extension, closed for modification.
- **Liskov Substitution Principle (LSP)**: Derived classes must be fully substitutable for their base classes.
- **Interface Segregation Principle (ISP)**: Clients should not be forced to depend on interfaces they do not use.
- **Dependency Inversion Principle (DIP)**: Depend on abstractions, not concrete implementations.

Quick Revision — Module 3

OOP Principles, Inheritance, Polymorphism, Packages & Interfaces | OE-EC704A

Inheritance, Packages & Interface Syntax

Concept	Syntax and Structural Format
Subclass Extension	<code>class Child extends Parent { ... }</code>
Super Constructor Call	<code>Child(int val) { super(val); ... }</code>
Interface Declaration	<code>interface Flyable { void fly(); }</code>
Interface Implementation	<code>class Bird implements Flyable { public void fly() { ... } }</code>
Package Syntax	Access and Import Declarations
Package Header	<code>package com.cgec.webtech;</code>
Import Header	<code>import com.cgec.webtech.MyClass;</code>
Abstract Method	<code>abstract void calculateScore();</code>
Constant in Interface	<code>int MAX_RETRY = 5; // Public static final by default</code>

One-Line Definitions

- **Inheritance:** The mechanism by which a subclass inherits variables and methods from a superclass.
- **Method Overloading:** Declaring methods with the same name but different parameters in one class.
- **Method Overriding:** Declaring a method in a subclass with the same signature as in the parent class.
- **Dynamic Method Dispatch:** Dynamic binding resolution that determines overridden method calls at runtime.
- **Abstract Class:** A class that cannot be instantiated and may contain abstract methods.
- **Interface:** A contract defining empty method signatures that implementing classes must satisfy.
- **Package:** A namespace group containing related classes, interfaces, and packages.
- **SOLID:** A set of five object-oriented design principles for readable and maintainable software.

Frequently Asked / Viva Questions

Q1. Why does Java not support multiple inheritance with classes?

To avoid the diamond problem. If class C extends both class A and class B, and both parents define a method with the same signature `show()`, class C cannot determine which parent version to inherit.

Frequently Asked / Viva Questions (contd.)**Q2. How does Java support multiple inheritance then?**

Java allows a class to implement multiple interfaces. Since interfaces do not hold instance state and their methods were traditionally empty, there is no duplicate implementation ambiguity.

Q3. What is the difference between `this()` and `super()` constructor calls?

`this()` invokes another constructor within the same class. `super()` invokes a constructor of the parent superclass. Both must be the first line of their respective constructors.

Q4. Can an abstract class have constructors? Why?

Yes. Although an abstract class cannot be instantiated directly, its constructor is called during subclass instantiation via `super()` to initialize its fields.

Q5. What is the difference between `final` class, `final` method, and `final` variable?

A `final` class cannot be inherited. A `final` method cannot be overridden. A `final` variable behaves as a constant and cannot be reassigned.

Q6. What is the contract between `equals()` and `hashCode()`?

If two objects are equal according to `equals()`, they must return the same integer hash code from `hashCode()`. If overridden, both must be updated to preserve this rule.

Q7. What is the difference between composition and inheritance?

Inheritance represents an IS-A relationship and creates tight coupling. Composition represents a HAS-A relationship, wrapping objects to provide loose coupling.

Q8. How does the access level of a subclass method override compare to the parent class method?

An overriding method cannot restrict the access level of the overridden parent method. For example, a `protected` method in a parent class can only be overridden as `protected` or `public`.

Q9. What are default methods in interfaces and why were they introduced in Java 8?

Default methods have an implementation inside the interface using the `default` keyword. They allow adding new methods to interfaces without breaking existing implementing classes.

Q10. What is the difference between compile-time and runtime polymorphism?

Compile-time polymorphism (overloading) resolves method calls during compilation using static binding. Runtime polymorphism (overriding) resolves method calls during execution using dynamic binding.

Java Access Modifier Visibility Matrix

Modifier	Same Class	Same Package	Subclass	Everywhere
private	Yes	No	No	No
Default	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes